Substitution Ciphers

If you ever had a secret decoder ring, it probably implemented a substitution *cipher*. The idea is pretty simple: you build a table that maps each character to another, such as that shown in Figure 13-2. You *encrypt* a message by replacing each original character with its counterpart from the table and *decrypt* by doing the reverse. The original message is called *cleartext*, and the encrypted version is called *ciphertext*.

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
q	s	a	o	z	w	e	n	y	d	p	f	c	x	k	g	u	t	m	v	l	b	r	h	j	i

Figure 13-2: Substitution cipher

This cipher maps *c* to *a*, *r* to *t*, *y* to *j*, and so on, so the word *cryptography* would be enciphered as *atjgvketqgnj*. The reverse mapping (*a* to *c*, *t* to *r*, and so on) deciphers the ciphertext. This is called a *symmetric* code, since the same cipher is used to both encode and decode a message.

Why isn't this a good idea? Substitution ciphers are easy to break using statistics. People have analyzed how often letters are used in various languages. For example, in English the most common five letters are *e*, *t*, *a*, *o*, *n*, in that order—or at least they were when Herbert Zim (1909–1994) published *Codes & Secret Writing* in 1948. Breaking a substitution cipher involves looking for the most common letter in the ciphertext and guessing that it's an *e*, and so on. Once a few letters are guessed correctly, it's easy to figure out some words, which makes figuring out other letters easy. Let's use the plaintext paragraph in Listing 13-1 as an example. We'll make it all lowercase and remove the punctuation to keep it simple.

```
theyre going to open the gate at azone at any moment
amazing deep untracked powder meet me at the top of the lift
```

Listing 13-1: Plaintext example

Here's the same paragraph as ciphertext using the code from Figure 13-2:

```
vnzjtz ekyxe vk kgzx vnz eqvz qv qikxz qv qxj ckczxv
qcqiyxe ozzg lxvtqapzo gkrozt czzv cz qv vnz vkg kw vnz fywv
```

Listing 13-2 shows the distribution of letters in the enciphered version of the paragraph. It's sorted by letter frequency, with the most commonly occurring letter at the top.

```
zzzzzzzzzzzzzzz
vvvvvvvvvvvvv
qqqqqqqqq
kkkkkkkk
xxxxxxx
ccccc
eeee
gggg
nnnn
ooo
ttt
yyy
ii
jj
ww
a
f
l
p
r
```

Listing 13-2: Letter frequency analysis

A code breaker could use this analysis to guess that the letter `z` in the ciphertext corresponds to the letter `e` in the plaintext, since there are more of them in the ciphertext than any other letter. Continuing along those lines, we can also guess that `v` means `t`, `q` means `a`, `k` means `o`, and `x` means `n`. Let's make those substitutions using uppercase letters so that we can tell them apart.

```
TnEjtE eOyNe TO OgEN TnE eATE AT AiONE AT ANj cOcENT
AcAiyNe oEEg lNTtAapEo gOroEt cEET cE AT TnE TOg Ow TnE fywT
```

From here we can do some simple guessing based on general knowledge of English. There are very few three-letter words that begin with *t* and end with *e*, and *the* is the most common, so let's guess that `n` means `h`. This is easy to check on my Linux system, as it has a dictionary of words and a pattern-matching utility; `grep '^t.e$' /usr/share/dict/words` finds all three-letter words beginning with a `t` and ending with an `e`. Also, there is only one grammatically correct choice for the `c` in `cEET cE`, which is `m`.

```
THEjtE eOyNe TO OgEN THE eATE AT AiONE AT ANj MOMENT
AMAiyNe oEEg lNTtAapEo gOroEt  MEET ME AT THE TOg Ow THE fywT
```

There are only four words that match `o□en`: `omen`, `open`, `oven`, and `oxen`; only `open` makes sense, so `g` must be `p`. Likewise, the only word that makes sense in `to open the □ate` is `gate`, so `e` must be `g`. There's only one two-letter word that begins with `o`, so `ow` must be `of`, making the `w` an `f`. The `j` must be a `y` because the words `and` and `ant` don't work.

```
THEYtE GOyNG TO OPEN THE GATE AT AiONE AT ANY MOMENT
AMAiyNG oEEP lNTtAapEo POroEt MEET ME AT THE TOP OF THE fyFT
```

We don't need to completely decode the message to see that statistics and knowledge of the language will let us do so. And so far we've used only simple methods. We can also use knowledge of common letter pairs, such as `th`, `er`, `on`, and `an`, called *digraphs*. There are statistics for most commonly doubled letters, such as `ss`, and many more tricks.

As you can see, simple substitution ciphers are fun but not very secure.

Transposition Ciphers

Another way to encode messages is to scramble the positions of the characters. An ancient transposition cipher system supposedly used by the Greeks is the *scytale*, which sounds impressive but is just a round stick. A ribbon of parchment was wound around the stick. The message was written out in a row along the stick. Extra dummy messages were

written out in other rows. As a result, the strip contained a random-looking set of characters. Decoding the message required that the recipient wrap the ribbon around a stick with the same diameter as the one used for encoding.

We can easily generate a transposition cipher by writing a message out of a grid of a particular size, the size being the key. For example, let's write out the plaintext from Listing 13-1 on an 11-column grid with the spaces removed, as shown in Figure 13-3. We'll fill in the gaps in the bottom row with some random letters shown in italics. To generate the ciphertext shown at the bottom, we read down the columns instead of across the rows.

t	h	e	y	r	e	g	o	i	n	g
t	o	o	p	e	n	t	h	e	g	a
t	e	a	t	a	z	o	n	e	a	t
a	n	y	m	o	m	e	n	t	a	m
a	z	i	n	g	d	e	e	p	u	n
t	r	a	c	k	e	d	p	o	w	d
e	r	m	e	e	t	m	e	a	t	t
h	e	t	o	p	o	f	t	h	e	l
i	f	t	a	s	d	f	g	h	i	j

tttaatehihoenzrrefeoayiamttyptmnceoareaogkepsenzmdetodgtoeedmffohnnepetgieetpoahhngaauwteigatmndtlj

Figure 13-3: Transposition cipher grid

The letter frequency in a transposition cipher is the same as that of the plaintext, but that doesn't help as much since the order of the letters in the words is also scrambled. However, ciphers like this are still pretty easy to solve, especially now that computers can try different grid sizes.